gram.en: An Explainable Grammatical Analysis Engine

Sinan Olsson-Pasic

June 11, 2026

Abstract

gram.en is an explainable grammatical analysis engine. It combines finite-state morphology, Earley parsing, feature-structure unification, and targeted error rules in order to diagnose not only whether an input sentence is grammatical, but which grammatical constraint failed and where. This paper develops the theory and the implementation together. The mathematical thread explains why the engine uses finite automata for morphology, a context-free backbone for syntax, and feature unification for local constraints. The engineering thread shows how those ideas become a compiler-like pipeline for natural language.

In an era when neural systems dominate grammatical error correction, a rule engine earns its place by what its verdicts *carry*. Every diagnosis **gram.en** emits names the violated constraint, the offending span, and a derivable repair — a structured, reproducible judgment rather than a post-hoc explanation inferred from a model's edit. Judgments of that shape can serve as verifiers and evaluation harnesses for statistical systems, and as generators of labeled error data, roles an unexplained correction cannot play. The engine measures its own claims accordingly: on a held-out probe set it reports precision of its error flags and its abstention rate on errors outside its declared scope, treating "out of coverage" as an answer in its own right.

The current grammars are deliberately small, but the implementation is not English-specific: the start symbol, tokenizer, and repair primitives are declared by the grammar, and a second Swedish fragment exercises the same machinery with different agreement and definiteness patterns.

Contents

1 Introduction

This paper documents the design and development of **gram.en**, an explainable grammatical analysis engine built from ideas in formal language theory, complexity theory, compiler design, and computational linguistics. The engine analyzes natural-language input through a compiler-like pipeline: tokenization, finite-state morphology, lexical lookup, parsing, feature-structure unification, and rule-level diagnostic reporting.

The motivating question was simple: *Can a natural language be represented as a set of executable rules?* Programming languages such as C, Python, and R are intentionally formal systems. Their syntax and behavior are specified, and compilers can reject invalid input according to fixed rules. Natural languages such as English, Swedish, and Russian are different. They are ambiguous, exception-heavy, historically layered, and only partially captured by explicit rules. Human speakers nevertheless internalize many of these patterns through exposure: we often judge whether a sentence "sounds right" without consciously naming the rule being applied.

gram.en attempts to make part of that implicit knowledge explicit. It treats grammatical analysis as a static-analysis problem over natural language. A sentence is tokenized, morphologically analyzed, parsed against a context-free backbone, and checked by unifying grammatical feature structures such as number, person, case, tense, and verb form. When a constraint fails, the engine attempts to report not only that the sentence is ungrammatical, but which rule failed, where it failed, and what repair follows from that failure.

This is the central design claim of **gram.en**: grammatical verdicts and grammatical explanations should come from the same computation. The system should not first reject a sentence and then search for an unrelated explanation after the fact. Instead, the failed parse, failed unification, or matched mal-rule should itself contain the information needed for the report.

Concretely, given the input `the dog bark`, the engine prints:

```
the dog bark
        ^^^^


Verdict:   ungrammatical

Violation: subject-verb agreement (number/person)
Rule:      S -> NP VP
Fix:       "the dog barks"  |  "the dog barked"  |  "the dogs bark"
```

Nothing in that report is generated after the fact. The violation is the feature equation that failed, the highlighted span is the constituent it failed on, and each suggested fix is a candidate the engine actually re-parsed and verified. Section 15 walks through the computation behind this report step by step; the rest of the paper builds the machinery it runs on.

The goal is not to replace modern statistical or neural grammar-correction systems. Those systems are far better suited to broad coverage, graded acceptability, and idiomatic language use. **gram.en** is instead a proof of concept for a different trade-off: limited coverage in exchange for explicit structure, predictable behavior, and explainable diagnostics.

That trade-off has, if anything, grown more relevant as neural systems have grown stronger. A correction produced by a neural model explains itself only through the diff between input and output; the reason has to be reconstructed afterwards, and the reconstruction is itself a guess. A

rule engine whose every verdict carries a named constraint, a span, and a verified repair produces judgments of a different kind: reproducible, inspectable, and structured. Such judgments can do work that an unexplained correction cannot. They can *verify* the output of a statistical system against the grammar it claims to respect; they can serve as an *evaluation harness* whose decisions do not drift between model versions; and because each diagnostic is a (sentence, constraint, span, repair) tuple, they can *generate labeled error data* for training and evaluating the very systems they complement. The point of the explicit machinery in this paper is not nostalgia for rules; it is that explanation-bearing judgments are a primitive that black-box systems still need and cannot supply for themselves.

How to read this document. Readers with a computer science background but no prior linguistics can treat the formal pumping-lemma statements and proofs (gathered in Appendix A) as optional; the informal treatment and its natural-language consequences in the main text are sufficient for understanding the engine's design rationale. The worked example in Section 15 demonstrates the engine's behavior concretely and can be read independently of the mathematical background. Readers who want to use or extend the engine should focus on Section 9 (Quick Start), the grammar file format (Section 21), and the performance section; the theoretical sections provide motivation but are not required for day-to-day grammar authoring. Researchers familiar with computational linguistics should begin with Section 2 (Related Work), Section 6.2 (decidability of feature-augmented parsing), and Section 14.4 (diagnostic soundness).

2 Related Work

The problem of explainable grammatical analysis has a prior history in both formal and applied computational linguistics. Several systems address overlapping goals; **gram.en** is positioned differently from each.

Grammatical Framework. Ranta's Grammatical Framework (GF) [10] is the closest architectural predecessor. GF separates an abstract grammar, which encodes language-independent structure, from concrete grammars that map it to surface forms in specific languages. This division of labour is functionally analogous to **gram.en**'s separation of the language-neutral engine from language-specific `.gram` files: one shared recognizer, multiple concrete rule sets. The key difference is the design goal. GF targets multilingual generation and correct parsing; it does not provide constraint relaxation, mal-rules, or localized diagnostic reporting. **gram.en** trades GF's generation capacity for a diagnostic-first pipeline in which the explanation is not inferred after a correction is produced but is instead the direct output of the failed computation.

HPSG and the LKB. Head-Driven Phrase Structure Grammar [9] makes typed feature structures a central organizing principle: every grammatical object carries a rich feature structure, and all constraints are stated as unification requirements. The Linguistic Knowledge Builder (LKB) [2] provides a grammar engineering workbench for HPSG. **gram.en** shares the feature-structure formalism but is narrower in scope. It does not adopt a full type hierarchy, does not target wide coverage, and adds diagnostic-specific machinery—constraint relaxation and mal-rules—that falls outside the HPSG engineering tradition.

LanguageTool. LanguageTool [6] is the most directly comparable *deployed* rule-based grammar checker. It operates primarily over token sequences using hand-written regular patterns augmented

by POS tags and shallow linguistic heuristics, which gives it broad practical coverage without deep parsing. **gram.en** instead builds full constituent structure via an Earley chart parser and enforces unification constraints over that structure. The trade-off is that **gram.en** covers a narrower fragment but can attribute failures to a specific named constraint rather than a matched surface-level pattern.

Neural grammatical error correction. Modern neural GEC systems, evaluated on benchmarks such as CoNLL-2014 [7] and BEA-2019 [1], achieve high recall over a wide range of error types. Their explanations, if any, are post-hoc: the model corrects a sentence, and the nature of the correction is inferred from the diff. **gram.en** does not compete on recall. Its goal is the complementary trade-off: a system where the explanation is not inferred after the fact but is the direct output of the computation that detected the failure.

3 Languages as sets of strings

A grammar engine needs a precise meaning for the statement "this sentence is valid." In ordinary speech, a language is a social and historical object. For a computer scientist, the first abstraction is simpler: a language is a set of strings. If Σ is an alphabet, then Σ^* is the set of all finite strings over that alphabet, and a language is any subset $L \subseteq \Sigma^*$.

This abstraction is deliberately austere. It ignores meaning, speakers, context, and style. But it gives the engine a clear decision problem: given an input string w, decide whether $w \in L$. A grammar is a finite description of such a set.

Informally, a grammar is a finite collection of rules that together describe which strings belong to the language. The formal tuple below names the four ingredients: the abstract categories used during construction, the surface symbols that appear in the final strings, the rules that rewrite one into the other, and the starting category from which every derivation begins.

Definition 3.1 (Generative grammar)**.** A generative grammar is a tuple

$$G = (V_N, V_T, R, S),$$

where V_N is a finite set of nonterminals, V_T is a finite set of terminals with $V_N \cap V_T = \emptyset$, R is a finite set of rewrite rules, and $S \in V_N$ is the start symbol. A rule is a pair (P, Q) with

$$P \in (V_N \cup V_T)^+, \qquad Q \in (V_N \cup V_T)^*.$$

A rule (P, Q) licenses one rewriting step. If P appears inside a larger string $\alpha P \beta$, then the rule rewrites that string as $\alpha Q \beta$. Write this one-step relation as $\Rightarrow$.

Definition 3.2 (Derivation)**.** The reflexive–transitive closure of $\Rightarrow$ is written $\Rightarrow^*$. Thus $\alpha \Rightarrow^* \gamma$ means that γ can be obtained from α by zero or more rule applications.

The distinction between terminals and non-terminals is the distinction between surface material and structure. *Terminals* are the symbols that may appear in the final generated string: for a natural-language grammar, these are usually words, punctuation marks, or token-level analyses. They are called terminal because derivation stops once only terminals remain. *Non-terminals* are abstract syntactic categories such as S, NP, VP, N, and V. They do not appear in the final sentence. Instead, they name intermediate structure that must still be expanded by grammar rules.

For example,

$$\text{S} \Rightarrow \text{NP VP} \Rightarrow \textsc{Det N V} \Rightarrow \textit{the dog barks}.$$

Here S, NP, VP, DET, N, and V are non-terminals, while *the*, *dog*, and *barks* are terminals.

The language generated by G is then

$$L(G) = \{\, w \in {V_T}^* \mid S \Rightarrow^* w \,\}.$$

where $L(G)$ is the set of all terminal strings w that can be derived from the start symbol S by applying rules from some grammar G some finite number of times. The expression ${V_T}^*$ means any finite string made only from terminal symbols, and $\Rightarrow^*$ means zero or more rewriting steps.

A derivation is a proof of membership. If $S \Rightarrow^* w$, then the grammar contains a witness that w belongs to the language. If no derivation exists, then w lies outside the language described by this particular grammar. That last phrase matters: outside $L(G)$ does not necessarily mean outside English. It may only mean outside the fragment that has been implemented.

3.1 Finite descriptions and infinite languages

Grammars are finite objects, but the languages they describe may be infinite. The grammar for balanced parentheses does not list every balanced string; it gives a recursive pattern that generates all of them. This is the first reason formal grammar is useful: finite rules can describe infinite regularities.

There is also a boundary. Not every language over an alphabet can be described by a finite grammar or program. There are only countably many finite strings that could serve as descriptions, but uncountably many subsets of Σ^*. Therefore most languages are not finitely describable at all. For this engine, the consequence is practical rather than philosophical: the goal is not to describe "all of English." The goal is to choose a useful, computable fragment whose rules can be enforced and explained.

4 The computational cost of grammar

A grammar formalism is not just a linguistic choice, but rather serves as an algorithmic budget. Once the engine chooses a grammar class, it also chooses what kind of recognizer it can use and how expensive membership will be.

The Chomsky hierarchy organizes grammars by the shape of their rewrite rules. Restricting rule shape restricts memory, and restricting memory changes what dependencies can be recognized.

Definition 4.1 (The four grammar types)**.** A grammar (V_N, V_T, R, S) is:

1. **Type 0**, or recursively enumerable, if there is no restriction on the rules.
2. **Type 1**, or context-sensitive, if rules are non-contracting: $|P| \leq |Q|$.
3. **Type 2**, or context-free, if every left-hand side is a single nonterminal: $A \to Q$.
4. **Type 3**, or regular, if every rule is right-linear, for example $A \to aB$ or $A \to a$.

The inclusions are strict:

$$\textsc{Reg} \subsetneq \text{CF} \subsetneq \text{CS} \subsetneq \text{RE}.$$

Type	Class	Recognizer	Memory resource
0	recursively enumerable	Turing machine	unbounded tape
1	context-sensitive	linear-bounded automaton	tape linear in input
2	context-free	pushdown automaton	one stack
3	regular	finite automaton	finite control only

Table 1: Grammar power tracks memory.

Regular languages are recognized with no unbounded memory. Context-free languages add one stack, enough to match nested dependencies. Context-sensitive languages allow more general bounded work over the input. Recursively enumerable languages allow arbitrary computation.

4.1 Membership as the engine's core decision problem

The direct question asked by a recognizer is membership: given a grammar G and input w, is $w \in L(G)$? The answer becomes more expensive as the grammar formalism becomes more powerful. At one extreme, a regular grammar needs only a finite counter and can be decided in a single linear scan of the input. At the other extreme, an unrestricted grammar can encode arbitrary computation, making the membership question unanswerable in general. The theorem below makes this precise.

Theorem 4.1 (Cost of membership)**.** *For a fixed grammar and input length n, regular membership is $O(n)$, context-free membership is decidable in $O(n^3)$ by standard dynamic programming methods such as CKY, context-sensitive membership is PSPACE-complete (meaning the hardest instances require space growing polynomially in the input length, which in practice means feasibility runs out of memory before it runs out of time), and unrestricted Type 0 membership is undecidable (meaning no algorithm terminates correctly on all inputs).*

Proof sketch. A deterministic finite automaton reads the input once, so regular recognition is linear. For context-free grammars in Chomsky normal form, CKY fills a triangular table of spans; there are $O(n^2)$ spans and $O(n)$ split points per span, giving $O(n^3)$ recognition for a fixed grammar. A linear-bounded automaton has only polynomially many configurations but may need polynomial space to search them, placing context-sensitive membership in PSPACE, with a matching lower bound. For unrestricted grammars, derivations can simulate Turing machines, so deciding membership would decide the halting problem. □

This theorem is one of the engine's design constraints. A fully general grammar is not merely slow; it is unanswerable. A practical grammar checker therefore has to live near the context-free level: expressive enough to build phrase structure, but restricted enough to keep recognition decidable and polynomial.

Definition 4.2 (Weak and strong generative capacity)**.** The *weak* generative capacity of a grammar is the string set $L(G)$. The *strong* generative capacity is the set of structures, such as parse trees, that the grammar assigns. Two grammars can be weakly equivalent while assigning different structures.

A yes/no grammar checker asks a weak-capacity question. An explainable grammar checker asks a strong-capacity question. To say that `the dog bark` violates subject–verb agreement, the engine

needs more than rejection; it needs the subject, the verb phrase, the rule that combined them, and the feature values that failed to unify.

5 Pumping lemmas and natural-language dependencies

The hierarchy says that machines with more memory can recognize more languages. But to show that the inclusions are genuinely strict, we need negative arguments: proofs that a language cannot be recognized by a weaker machine. Pumping lemmas provide those tests.

A pumping lemma says that every sufficiently long string in a language from a given class contains a repeatable part. For regular languages, the repeatable part comes from a finite automaton revisiting a state. For context-free languages, the usual proof finds the repeatable material by observing that a sufficiently tall parse tree must repeat a nonterminal along a root-to-leaf path. In both cases, the machine has used some bounded resource twice, so some corresponding part of the string can be duplicated or deleted while staying in the language.

5.1 One stack short: nested dependencies

For regular languages the lemma reads, in words: every sufficiently long string in the language contains a nonempty middle piece that can be repeated or deleted while the resulting string stays in the language. The repeatable piece exists because a finite automaton reading a long string must revisit some state, and whatever it read between the two visits can be traversed again, or skipped, without the automaton noticing.

The standard counterexample is the language of matched blocks,

$$L = \{a^n b^n \mid n \geq 0\} : \qquad \varepsilon,\ \texttt{ab},\ \texttt{aabb},\ \texttt{aaabbb},\ \ldots$$

Take a long string $a^p b^p$. The pumpable middle piece must sit near the start of the string — among the a's — so pumping changes the number of a's while leaving the b's fixed, and the result falls out of the language. No finite automaton recognizes L. (The formal statement and proof are in Appendix A.)

The point is not the letters a and b. The point is the dependency shape. The language $\{a^n b^n\}$ requires matching two unbounded counts. A finite automaton has no stack on which to remember how many a's it saw. Center embedding in natural language has this nested shape: "the rat [the cat [the dog chased] killed] ate" contains dependencies that must be closed in reverse order. This is why a purely finite-state syntax model cannot capture the full range of unbounded nested dependencies.

5.2 Two stacks short: crossing dependencies

One stack can handle nesting, but it cannot handle every kind of dependency. In particular, it cannot generally compare three independent counts or copy an arbitrary string.

The context-free version of the lemma finds *two* linked pieces that must pump together. They come from a repeated nonterminal along a path in a tall parse tree: duplicating the material between its two occurrences duplicates two substrings at once, one on each side. The standard counterexample is

$$L = \{a^n b^n c^n \mid n \geq 0\},$$

which requires three blocks to stay the same length. The two linked pieces of any pumping decomposition can touch at most two of the three blocks, so pumping breaks the three-way equality. One

stack can match one unbounded dependency, as in $\{a^n b^n\}$; it cannot keep three counts synchronized. (Formal statement and proof sketch: Appendix A.)

This matters because some natural-language dependencies are not purely nested. Shieber's Swiss German argument uses cross-serial dependencies [12]: a sequence of noun phrases is followed by a sequence of verbs, and each verb is linked to the corresponding noun phrase in the *same* left-to-right order rather than the reverse order that nesting would impose. In English, nested relative clauses close in last-in-first-out order (the most recently opened dependency closes first). In Swiss German verb clusters, the pairing goes in parallel: the first noun phrase controls the first verb, the second controls the second, and so on. A single stack cannot handle this because its last-in-first-out discipline can only track one unbounded dependency at a time, not two simultaneously unfolding sequences. (The formal reduction to a language of the shape $\{a^m b^n c^m d^n \mid m, n \geq 0\}$, which is demonstrably not context-free, is given in Appendix A.)

Theorem 5.1 (Shieber 1985, informal [12])**.** *Some natural-language string sets are not context-free.*

The engineering consequence is conservative. Natural language is not strictly context-free, but the constructions that witness this are not the primary source of everyday grammar-checking errors. **gram.en** therefore uses a context-free backbone plus feature constraints. It deliberately defers mildly context-sensitive phenomena in exchange for a simpler, polynomial, explainable core.

Definition 5.1 (Mild context-sensitivity)**.** A class of languages is mildly context-sensitive if it properly contains the context-free languages, captures limited cross-serial dependencies, has semilinear growth, and remains parsable in polynomial time.

Tree-Adjoining Grammar, Combinatory Categorial Grammar, Linear Indexed Grammar, and Head Grammar are weakly equivalent in the sense of Vijay-Shanker and Weir. This is a useful landmark, but not the target of the current engine. The current engine chooses a practical point below that line: finite-state morphology, context-free phrase structure, and unification constraints.

6 From generation to constraints

The grammars above are generative: start from S and rewrite until a terminal string appears. For diagnosis, it is often more useful to invert the viewpoint. A grammar can be treated as a set of constraints that a candidate structure must satisfy. Grammaticality becomes the existence of a structure for the input whose constraints are all consistent.

This is the step that turns recognition into explanation. The same constraint system that accepts a well-formed structure also identifies the point of failure in an ill-formed one. A failed constraint is local: it mentions particular constituents. It is also named: it is a particular agreement, case, or selection rule. That gives the engine something to report.

6.1 Feature structures

Natural-language rules often talk about properties rather than just categories. It is not enough to know that a word is a noun or a verb. The engine may also need to know whether a noun is singular or plural, whether a pronoun is nominative or accusative, or whether a verb is finite, bare, past-tense, or participial.

A *feature structure* is the object used to store those grammatical properties. At the simplest level,

it can be read like a small record, dictionary, or table of labelled facts:

$$[\textsc{cat} : \text{N},\ \textsc{num} : \textsc{sg},\ \textsc{pers} : 3,\ \textsc{case} : \textsc{nom}].$$

This says that the item is a noun, singular, third person, and nominative. In a programming-language compiler, a parser may know that an expression is an `Expr`, while later phases attach extra information such as its type, scope, or inferred value category. Feature structures play the same role here: the parser may know that a phrase is an NP, but the feature structure records the extra grammatical facts needed to decide where that phrase can legally appear.

For example, the phrase *the dog* is not merely an NP. It is an NP with agreement information:

$$[\textsc{cat} : \text{NP},\ \textsc{num} : \textsc{sg},\ \textsc{pers} : 3].$$

That extra information is what lets the sentence rule reject *the dog bark*: the subject is third-person singular, while the verb form *bark* does not license that subject.

Definition 6.1 (Feature structure)**.** A feature structure is a finite directed acyclic graph with labelled feature arcs and atomic values at leaves. Equivalently, it is a partial map from feature paths to values.

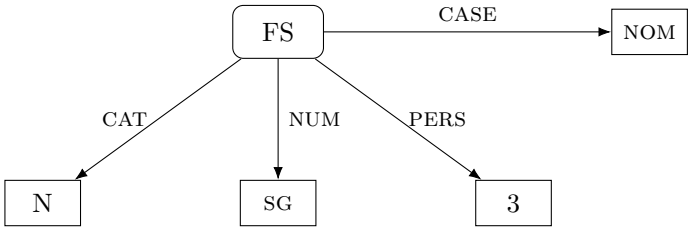

Figure 1: A flat feature structure represented as a directed acyclic graph.

The same object can be read either as a graph or as the attribute–value record $[\textsc{cat} : \text{N}, \textsc{num} : \textsc{sg}, \textsc{pers} : 3, \textsc{case} : \textsc{nom}]$.

For a computer scientist, a feature structure can be understood as a data structure for grammatical facts. In the simplest case it behaves like a record or dictionary: feature names are keys, and grammatical values are entries.

The formal graph-based definition is more general than the flat record notation. It allows nested information and shared substructure. For the examples in this engine, however, the record notation is usually enough: a feature structure is a bundle of grammatical facts carried by a word or phrase.

Two feature bundles are compatible if they never disagree on any specific value; a missing value is not a disagreement. Unification is the operation that merges compatible bundles into one, and fails immediately when they conflict. The formal definition below gives this a precise order-theoretic meaning.

Definition 6.2 (Subsumption and unification)**.** A feature structure F subsumes G, written $F \sqsubseteq G$, when G contains all the information in F and possibly more. The unification $F \sqcup G$ is the least structure that contains the information from both. If the two structures demand incompatible atomic values on the same path, unification fails.

Subsumption is an information-ordering. A less specific structure can describe more possibilities:

$$[\textsc{num} : \textsc{sg}]$$

says only that something is singular. A more specific structure,

$$[\textsc{num} : \textsc{sg},\ \textsc{pers} : 3,\ \textsc{case} : \textsc{nom}],$$

contains that information and adds more. Therefore the first structure subsumes the second.

Unification is the operation that combines compatible grammatical information. It behaves like merging two records, except that a conflict on the same field is an error:

$$[\textsc{num} : \textsc{sg}] \sqcup [\textsc{pers} : 3] = [\textsc{num} : \textsc{sg},\ \textsc{pers} : 3].$$

But incompatible values cannot both be true:

$$[\textsc{num} : \textsc{sg}] \sqcup [\textsc{num} : \textsc{pl}] = \top.$$

The symbol $\top$ denotes inconsistency. It is not an ordinary feature structure; it is an adjoined failure value. The convention here is that conflicting information rises to $\top$: a subject requiring singular agreement and a verb requiring plural agreement cannot both be satisfied in the same structure.

Operationally, unification is the grammar engine's type-checking step. Compatible information merges and continues upward through the parse tree. Incompatible information produces a local failure.

Theorem 6.1 (Unification as a join)**.** *If inconsistent structures are represented by a top element $\top$, then unification is the least upper bound operation in the subsumption order. It succeeds exactly when the two feature structures have a common refinement.*

Proof sketch. Compatible feature information can be merged path-by-path. If two paths assign the same atomic value, they agree. If one path is absent, the present value is retained. If both paths assign distinct atomic values, no structure can satisfy both requirements, so the result is $\top$. When no conflict occurs, the merged structure contains all the information from both inputs and adds nothing beyond what one input already required. It is therefore the least structure above both inputs in the subsumption order. □

For the engine, this is the mechanism that makes explanations possible. A parse failure caused by unification is not a bare rejection. It has a rule, two constituents, a feature path, and two incompatible values. The same computation that rejects an analysis also tells the engine what grammatical constraint was violated.

6.2 Decidability of feature-augmented parsing

Adding feature unification to a context-free backbone does not automatically preserve polynomial recognition. The decidability and complexity of the augmented system depend on the feature logic.

Kasper and Rounds [5] show that the full logic of feature structures—allowing re-entrant paths (two distinct paths sharing the same substructure node) and infinite value domains—is equivalent in expressive power to first-order logic over feature trees. Membership for grammars that use unrestricted feature logic is therefore potentially undecidable.

The engine avoids this by design. Every feature declaration in a `.gram` file enumerates a finite closed value domain:

```
%feature num  : sg | pl
%feature pers : 1 | 2 | 3
%feature vform: bare | fin | pastp | presp
```

With finite value domains, unification is bounded per rule application. The number of distinct feature structures for a fixed grammar is finite, so the augmented Earley chart remains finite and recognition stays polynomial in the input length.

One declaration form is open: `%feature lemma : *` does not enumerate its values. Finiteness survives, but by a different argument: every value an open feature can take originates in the lexicon or in a rule equation, both of which are fixed at grammar load time, so the set of values that can actually occur in a parse is still finite for a fixed grammar. The closed-domain argument bounds the feature space by the declarations alone; the open-domain argument additionally leans on the lexicon being finite.

Re-entrancy is impossible by the data structure definition. The implementation represents a feature structure as a recursive `Map<string, feature_struct>` tree. Every `unify` call allocates a fresh `Map`, copying values path-by-path; no operation introduces shared nodes between two distinct paths. The structure is a tree, not a DAG. The polynomial bound therefore needs no caveat for re-entrancy.

One honest qualification remains, and it concerns the constant rather than the exponent. The implementation packs chart items by rule, dot position, origin, and the feature signatures of the daughters parsed so far, and it returns explicit derivation trees rather than a packed forest. Recognition is therefore polynomial in the input length for a fixed grammar, but the grammar-dependent constant is the number of distinct daughter-signature tuples, which is finite yet potentially much larger than the rule count. Reaching the textbook Earley constant would require packing items by the mother's feature structure alone and returning a shared parse forest; the engine trades that constant for direct access to the trees its diagnostics walk.

Grammar authors who stay within the declared feature vocabulary therefore get a polynomial-time recognizer. The feature declaration system, described in Section 21, enforces this at grammar load time: a value not in the declared domain is a load-time error, not a silent mismatch.

This places **gram.en** strictly below the mildly context-sensitive class (Tree-Adjoining Grammar, Combinatory Categorial Grammar) by design. Those formalisms handle cross-serial dependencies of the kind discussed in Section 5. **gram.en** defers those constructions in exchange for a simpler, polynomial, and fully explainable core.

7 Gradience and the coverage trap

The formal model above is crisp: either a string belongs to $L(G)$ or it does not. Natural language is not always crisp. Speakers make graded judgments using labels like acceptable, awkward, questionable, or unacceptable. This is one reason modern grammar correction often uses statistical and neural models: those models are much better at broad coverage and graded preferences. Take for instance the sentence `A big house barks`. While technically grammatically correct, the engine has no way of grading whether or not it makes sense for a house to bark, while a human would definitely question where you bought it.

The more dangerous issue for a rule engine is coverage. A hand-written grammar G defines only its implemented fragment. Therefore

$$\neg(w \in L(G)) \neq \text{“}w \text{ is ungrammatical.”}$$

No parse may mean that the sentence is wrong. It may also mean that the grammar has not learned the construction yet. Treating every failed parse as an error manufactures false positives.

gram.en therefore separates four outcomes:

1. **Grammatical.** A full parse succeeds with no feature violations: the sentence is grammatical within the implemented fragment.
2. **Ungrammatical.** A known constraint fails, a mal-rule matches, or a relaxed parse identifies a localized violation: the engine reports the named constraint.
3. **No analysis.** No parse succeeds and no diagnostic fires: the input is out of coverage. The engine explicitly does not claim the sentence is ungrammatical.
4. **Unknown word.** A token has no morphological analysis at all: the engine withholds any grammatical verdict rather than misattributing the failure.

This is a design rule, not just a philosophical caution. The engine should only report an error when it can point to the rule that was violated.

8 The engine as a compiler-like pipeline

The implementation follows the same staged discipline as a compiler front end. Each stage transforms one representation into another and passes ambiguity forward until later constraints can resolve it.

```
text -> tokenizer -> morphology -> lexicon -> parser(+unification)
                    -> error detection -> report
```

Stage	Responsibility
Tokenizer	Converts raw text into tokens, including contractions such as `don't -> do + n't`.
Morphology	Maps surface forms to possible lemmas, categories, and inflectional features.
Lexicon	Attaches categories and feature structures to closed-class words and analyzed stems.
Parser	Builds constituent structure using an Earley chart parser over a context-free backbone.
Unification	Enforces agreement, case, verb-form, and selection constraints.
Diagnostics	Converts localized failures and mal-rule matches into human-readable reports.

Table 2: The analysis pipeline.

The pipeline is language-independent in shape. The language-specific data live in the lexicon, the morphology, and the grammar file. Ambiguity enters early: `bark` may be a noun or a verb, and

`barked` may be finite past or a past participle. The parser keeps competing analyses alive and lets structure decide which ones fit.

Later sections unpack these data sources separately: phrase-structure rules live in the grammar file, closed-class items live in the lexicon, and productive inflection is compiled from continuation lexicons and rewrite rules.

9 Quick Start

The engine is written in TypeScript and runs on Node.js 22.6.0 or later using the runtime's native `--experimental-strip-types` flag, which executes TypeScript source files directly without a compilation step. For browser deployment the engine is bundled by esbuild into a single IIFE file that exposes a `GrammarEngine` global.

Command-line usage.

```
# Analyze a sentence (English grammar by default)
node --experimental-strip-types src/cli.ts "the dog bark"

# Select a language
node --experimental-strip-types src/cli.ts --lang sv "hunden skäller"

# Include the Earley chart trace (narrated to stderr)
node --experimental-strip-types src/cli.ts --trace "the dog bark"
```

Programmatic API (Node.js).

```
import { load_grammar } from "./src/languages.ts";
import { analyze }      from "./src/analyze.ts";

// Load once; the grammar compiles morphology and indexes the lexicon.
// Reuse the result across many sentences.
const g = load_grammar("en");   // also "sv"

const result = analyze(g, "the dog bark");
// result.verdict === "ungrammatical"
```

The `analysis` result type. The structured result corresponds to the four outcomes described in Section 7. Its TypeScript definition is:

```
type verdict = "grammatical"
             | "ungrammatical"
             | "no-analysis"
             | "unknown-word";

type reported_violation = {
    message   : string;          // human-readable constraint name
    rule      : string;          // e.g. "S -> NP VP"
    span      : [number, number];     // token range [i, j)
    char_span : [number, number];     // character range in source
    fixes     : string[];        // suggested corrections
};
```

```
type analysis = {
    verdict       : verdict;
    tokens        : string[];
    tree          : tree | null;
    violations    : reported_violation[];  // may be multiple
    unknown_words : string[];
};
```

For the input `the dog bark`, the result fields correspond directly to the pretty-printed report shown in Section 15:

```
{
  verdict    : "ungrammatical",
  tokens     : ["the", "dog", "bark"],
  violations : [{
    message  : "subject-verb agreement",
    rule     : "S -> NP VP",
    span     : [2, 3],      // "bark" occupies tokens [2, 3)
    char_span: [8, 12],
    fixes    : ["the dog barks", "the dogs bark"]
  }],
  unknown_words: []
}
```

The `violations` field is always an array. A sentence with more than one diagnosable constraint failure can carry multiple entries, one per distinct (rule, span) pair after deduplication.

Trace mode. The optional `trace` field in `analyze_options` is a callback `(line: string) => void`. When provided, the engine calls it once per chart event—predict, scan, complete, scan-miss, feature clash—narrating the Earley parse to that sink. The CLI passes `console.error` so that the trace appears on `stderr` while the verdict appears cleanly on `stdout`. Trace mode is described further in Section 18.

Browser demo. The same engine runs unmodified in the browser. A single-page demo (`index.html`) presents a text field; as the user types, the sentence is re-analyzed and any violation span is underlined in place. Clicking an underline opens a popover showing the violation message, the rule that fired, and the verified fix suggestions as buttons; clicking a fix applies it to the field and re-analyzes. A language switcher toggles between the English and Swedish grammars. The page is fully client-side: at build time each grammar is flattened (all `%include`/`%import` directives resolved) and bundled into one JavaScript file, so the demo needs no server and analyses run locally in a few milliseconds. The demo is the diagnostic pipeline made tangible—the underline is the violation's character span, and the popover's buttons are the repair candidates the engine re-parsed.

10 Tokenization

Tokenization maps a character string to a sequence of word-like units. For space-delimited languages this can look trivial, but it is still a linguistic decision. Contractions, clitics, punctuation, hyphenation, capitalization, and numbers all need policy.

In the current English fragment, contraction splitting is essential:

$$\texttt{don't} \mapsto \texttt{do} + \texttt{n't}, \qquad \texttt{I'm} \mapsto \texttt{I} + \texttt{'m}.$$

The clitic itself is not discarded. It becomes a token with lexical and syntactic behavior. This allows the grammar to distinguish `I don't bark`, `he doesn't bark`, and `me'll bark` using the same syntactic machinery that checks agreement and case elsewhere.

For languages without spaces, such as Japanese, tokenization becomes a major analysis problem in its own right. The current engine does not solve that problem, but it now exposes the seam explicitly. A grammar chooses a tokenizer with `%tokenizer`; tokenizer-specific parameters, such as English clitics, are declared with directives such as `%clitic`. The Unicode letter and number classes used by the whitespace tokenizer already cover characters such as `å`, `ä`, `ö`, and Cyrillic letters, so the first multilingual pressure point was not character recognition but language specific token policy.

11 Finite-state morphology

Morphology is where word forms are decomposed into stems and inflectional information. English has relatively light morphology, but even English needs productive patterns:

$$\texttt{dogs} \mapsto \texttt{dog + N + Pl}, \qquad \texttt{liked} \mapsto \texttt{like + V + Past}, \qquad \texttt{tried} \mapsto \texttt{try + V + Past}.$$

Listing every inflected form by hand works for a toy grammar and collapses immediately when the lexicon grows. The right abstraction is finite-state morphology.

11.1 Finite-state transducers

A finite-state automaton recognizes a language: it accepts or rejects a string. A finite-state transducer recognizes a relation between strings: it reads one string and writes another. An arc is labelled $a : b$, meaning read a on the input tape and write b on the output tape. The empty symbol ε allows insertion or deletion.

Definition 11.1 (Finite-state transducer)**.** A finite-state transducer is a tuple

$$T = (Q, \Sigma, \Gamma, E, q_0, F),$$

where Q is a finite state set, Σ is the input alphabet, Γ is the output alphabet, $q_0 \in Q$ is the start state, $F \subseteq Q$ is the accepting set, and $E \subseteq Q \times (\Sigma \cup \{\varepsilon\}) \times (\Gamma \cup \{\varepsilon\}) \times Q$ is the arc relation.

The engine uses transducers in both directions. In generation, a lexical analysis produces a surface word. In analysis, a surface word recovers possible lexical analyses. This symmetry is one reason finite-state methods are standard in morphology.

11.2 Continuation lexicons and trie compilation

The morphology compiler uses a LEXC-like idea: roots point to paradigms, and paradigms list the forms that a root can take.

```
%lex Root
    dog  : N-reg <lemma>=dog
    cat  : N-reg <lemma>=cat
```

```
    bark : V-reg <lemma>=bark

%lex N-reg : N
    +N+Sg : 0 <num>=sg <pers>=3
    +N+Pl : s <num>=pl <pers>=3

%lex V-reg : V
    +V+Inf  : 0  <vform>=bare
    +V+Past : ed <vform>=fin <tense>=past
```

The example writes roots inline for readability. In the implemented grammars, ordinary open-class stems are mostly imported from tabular lexicon files. The grammar still defines the paradigms; the TSV file supplies many roots that point into those paradigms. This keeps productive morphology declarative while keeping large stem lists out of the grammar source.

Adjectives are deliberately kept in the language grammar rather than moved into the shared imported stem table. They carry language-specific behavior: English degree morphology and suppletion, Swedish gender and definiteness agreement, and future language-specific adjective classes are not just bare open-class roots. Keeping them near the grammar rules makes those dependencies visible.

A naive implementation builds one path per root-form pair. The current engine instead builds a trie over stems and shares paradigm sub-FSTs. If many words share prefixes, the trie collapses common structure. If many roots share a paradigm, the continuation transducer is built once.

The state-count effect is visible in benchmark data:

stems	states	arcs	build	states/stem
100	514	734	0.5 ms	5.14
1,000	4,542	6,563	5.6 ms	4.54
10,000	38,109	58,130	86 ms	3.81
50,000	164,764	264,785	505 ms	3.30

Table 3: Trie prefix-sharing reduces states per stem as the lexicon grows.

The mathematical relation recognized by the transducer is unchanged. The implementation is smaller because equivalent prefixes and paradigm continuations are shared.

11.3 Replace rules

Concatenating a root and suffix is not always enough. English past tense illustrates the problem:

$$\texttt{like + ed} \mapsto \texttt{liked}, \qquad \texttt{try + ed} \mapsto \texttt{tried}.$$

Morphophonemic spelling rules mediate between an ideal lexical representation and the surface form.

A replace rule has the abstract form

$$A \to B \;|||\; L\_R,$$

In words, rewrite A as B in the context where L appears to the left and R to the right. Each rule denotes a regular relation, so a cascade of rules can be represented by composed finite-state transducers [4].

In the grammar language, character classes make these rules readable:

```
%class Vowel : a | e | i | o | u
%class Cons  : b | c | d | f | g | h | j | k | l | m | n | p | r | s | t | v | w | x | y
    | z

%rule e:0 => Cons _ e d
%rule y:i => _ e d
```

These rules allow `liked` and `tried` to be analyzed productively instead of being listed as one-off lexical entries.

11.4 Input-ε elimination

A transducer may contain arcs that consume no input. They are mathematically convenient, but can introduce challenges to work with in practice. A depth-first application algorithm can spend exponential time walking many different ε-paths that reach the same state and input position.

The standard fix is input-ε elimination. For every state s, compute its input-ε closure: the states reachable from s using arcs whose input label is ε, together with the output strings accumulated along those paths. For every real input-consuming arc leaving a state in that closure, install an equivalent arc directly from s whose output includes the accumulated closure output.

Formally, if

$$s \xrightarrow{\varepsilon:w} t \quad \text{and} \quad t \xrightarrow{x:y} u,$$

then the rebuilt transducer contains an arc

$$s \xrightarrow{x:wy} u.$$

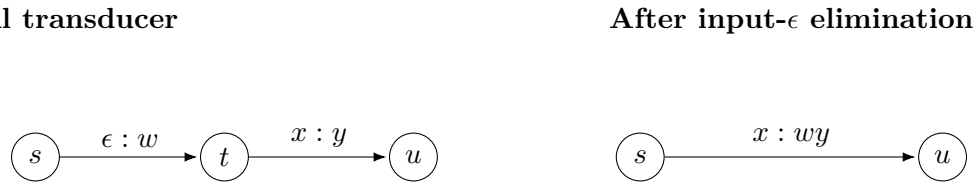

Figure 2: Input-ϵ elimination. A path that first follows an input-ϵ arc and then consumes a real input symbol is replaced by a single direct arc whose output accumulates both outputs.

Residual accepting paths through input-ε arcs are represented as accepting output at the state. Every run of the original machine factors into input-ε segments followed by input-consuming arcs, so every run has a corresponding run in the rebuilt machine, and conversely. The recognized relation is preserved.

One restriction makes the implementation simpler than the fully general construction: the closure keeps a single accumulated output per reachable state, so equivalence requires that between any two states, all input-ε paths carry the same output. The transducers the engine builds satisfy this by

shape—stem tries and continuation branches are trees—and the rewrite-rule compiler rejects pure-insertion rules (`0:x`), the one rule form whose compilation would create output-emitting input-ε cycles. An implementation that admits insertion rules would have to compute closures as sets of (state, output) pairs rather than a single output per state.

For the engine, the payoff is direct: after elimination, every path step that matters consumes input. The search depth is bounded by the length of the word rather than by the number of hidden ε-paths in the graph.

The same machinery extends to adjectives. A regular adjective paradigm derives positive, comparative, and superlative forms:

`small` $\mapsto$ `smaller`, `smallest`, `happy` $\mapsto$ `happier`, `happiest`, `large` $\mapsto$ `larger`, `largest`.

The spelling rules are not adjective-specific hacks. Silent-`e` deletion, `y`$\to$`i`, and consonant doubling are morphophonemic rules in the rewrite cascade. The lexicon then filters the cascade by requiring the derived surface form to correspond to a known root and paradigm.

12 Parsing with an Earley chart

Morphology gives possible word analyses. Syntax must decide how those analyses combine. The engine uses an Earley parser [3] because it can handle arbitrary context-free grammars, preserve ambiguity, and expose a useful chart trace.

An Earley item records a partially recognized production:

$$[A \to \alpha \bullet \beta, i, j],$$

meaning that the parser began recognizing A at position i, has recognized α up to position j, and still expects β.

The algorithm has three operations:

1. **Predict.** If the dot is before a nonterminal B, add items for productions of B.
2. **Scan.** If the dot is before a terminal that matches the next token, advance the dot.
3. **Complete.** If an item finishes a constituent, advance any earlier item that was waiting for that constituent.

In a purely context-free parser, completion only checks category symbols. In **gram.en**, completion also applies feature equations. If the rule's equations unify successfully, the completed constituent carries the merged feature structure. If unification fails, the item is blocked and its failure can become a diagnostic candidate.

In-memory representation. An Earley item $[A \to \alpha \bullet \beta, i, j]$ is stored as a record with four fields: the grammar rule, the dot position within that rule's right-hand side, the start position i, and the current end position j. Feature structures travel with *completed* constituents rather than with active items: when a completion step merges two constituents, their feature structures are unified and the result is attached to the newly completed item. An active item therefore carries only a rule reference and two positions; feature information is not materialized until completion succeeds and unification is checked.

Unification is non-destructive: every `unify` call returns a new `Map`, leaving both operands unchanged. There is no trailing or undo stack; the copy cost is bounded by the size of the feature structure, which is small for the current grammar's fixed feature vocabulary. The chart is stored as an array of columns indexed by end-position j, each column holding a `Map<string, item>` keyed by a stable serialization of the item so that cheaper derivations of the same dotted rule replace more expensive ones during relaxed parsing.

13 Feature equations in grammar rules

A phrase-structure rule builds categories. A feature equation states what must be true about their grammatical properties.

```
S -> NP VP
    <NP agr> = <VP agr> ! "subject-verb agreement" fix: agree-verb

NP -> Det N
    <NP agr> = <N agr>

VP -> V
    <VP agr> = <V agr>
```

The context-free backbone says that a sentence can consist of an NP followed by a VP. The feature equation says that this is only a valid sentence if their agreement features are compatible. Thus the parser and the constraint checker are not separate systems; constraint checking happens while parse structure is built.

One addressing rule matters even in small grammars. If a production contains the same category twice—coordination and ditransitives make this unavoidable, as in `NP -> NP Conj NP`—a bare path such as `<NP num>` no longer names a unique constituent. The format requires an alias there (`left:NP`, `right:NP`) and rejects the ambiguous bare name at load time. Section 21.3 gives the full resolution rules.

14 Diagnostics: from failure to explanation

The coverage trap says that no parse is not enough. The engine must only report errors it can localize. **gram.en** uses two diagnostic mechanisms.

14.1 Constraint relaxation

If strict parsing fails, the parser can retry while relaxing one tagged equation. If relaxing exactly that equation allows a full parse, then the equation is a plausible diagnosis.

For example, the rule

```
S -> NP VP
    <NP agr> = <VP agr> ! "subject-verb agreement" fix: agree-verb
```

names the agreement constraint. If `the dog bark` fails strictly but succeeds when this constraint is relaxed, the engine reports subject–verb agreement rather than a generic parse failure.

14.2 Repair primitives

A diagnostic explains why a parse failed; a repair strategy proposes nearby strings that would avoid the failure. Early versions of the engine treated repair names as English-specific hooks such as `agree-verb`. That made the diagnostic layer less declarative than the grammar layer: the rule could name a failure, but the engine still had to know what kind of constituent a noun, verb, determiner, or pronoun was.

The current format treats repairs as parameterized primitives:

```
fix: agree(V, num=sg) | swap(Pron, case=acc) | insert(Det)
```

The primitive names are generic. `agree` searches for an analyzed form in the same category whose features satisfy the requested constraint. `swap` replaces one category-compatible item with another item carrying different features. `insert` proposes a missing constituent of the named category. Literal fixes and reorder fixes remain special cases, but ordinary agreement, case, and determiner repairs no longer depend on English category names being hardcoded into the analyzer.

14.3 Mal-rules

Some mistakes are better recognized directly. A mal-rule is an intentionally wrong pattern with an attached diagnostic and repair.

```
%mal S -> V NP
    *ERR "word order: the verb appears before its subject"
```

The repair is not written by hand. When a mal-rule matches, the engine permutes the matched constituents and re-parses each ordering, reporting the reorderings that parse cleanly (`barks the dog` → `the dog barks`). The diagnostic message is authored; the fix is derived.

Mal-rules are precise but anticipatory: the grammar author has to know the error pattern in advance. Constraint relaxation is more general but can over-trigger when many repairs are possible. A practical engine uses both and ranks minimal, localized explanations above broad ones.

14.4 Soundness and completeness of diagnostics

The engine's diagnostic claims reduce to two properties: *soundness* (reported errors are real) and *completeness* (declared errors are found when present).

Soundness of constraint relaxation. When the engine reports a named constraint, it means that equation genuinely failed, on the reported constituents, inside an otherwise complete parse. This is structurally sound relative to the declared grammar: the constraint really did prevent strict parse completion. It is not sound in a deeper linguistic sense—the reported constraint set may not be the unique explanation for the observed surface form, particularly when distinct relaxations license the same string.

Soundness of mal-rule matching. A mal-rule fires when the ungrammatical pattern it encodes matches the input exactly. The matched pattern is the stated explanation. This is sound by definition: the rule was written to recognize precisely that error. The caveat is anticipatory: mal-rules require the grammar author to enumerate known error patterns in advance.

Completeness. The engine is not complete with respect to all possible English errors. It is complete only with respect to errors that the grammar declares: a sentence that violates a named constraint will be caught if no other parse succeeds. Errors outside the declared constraint set produce an out-of-coverage result rather than a false negative in the standard GEC sense. The coverage trap (Section 7) is the correct behavior here, not a failure mode.

Multiple errors and minimality. Relaxation is not one-constraint-at-a-time. The relaxed parse allows *every* diagnostic-tagged equation to record a violation instead of failing, and the chart keeps, per constituent, a derivation with the fewest accumulated violations. The engine then reports all violations carried by a minimum-violation full parse, so a sentence with two independent agreement errors receives both diagnoses from a single relaxed pass. Two limitations follow from the design rather than from cost. First, only tagged equations relax: a sentence rescued only by ignoring an *untagged* structural constraint is reported as out of coverage, not diagnosed. Second, minimality is the tie-break among competing explanations: when different violation sets license the same string, the engine reports a smallest one, which is usually but not necessarily the linguistically intended reading.

15 Worked example: `the dog bark`

Consider the input:

`the dog bark`

The tokenizer returns three tokens. The lexicon and morphology produce (vagr is the verb-agreement feature; its two values are 3sg, third-person singular, and n3sg, everything else):

$$\begin{aligned} \texttt{the} &\mapsto \mathsf{Det}, \\ \texttt{dog} &\mapsto \mathsf{N}[\mathsf{num} = \mathsf{sg},\ \mathsf{pers} = 3,\ \mathsf{vagr} = \mathsf{3sg}], \\ \texttt{bark} &\mapsto \mathsf{V}[\mathsf{vform} = \mathsf{fin},\ \mathsf{vagr} = \mathsf{n3sg}] \quad \text{or} \quad \mathsf{V}[\mathsf{vform} = \mathsf{bare}] \quad \text{or} \quad \mathsf{N}[\mathsf{num} = \mathsf{sg},\ \ldots]. \end{aligned}$$

The parser tries the finite verbal reading of `bark`. It builds an NP from `the dog`; that NP inherits the noun's third-person-singular agreement. It builds a VP from `bark`; the finite present form `bark` is the one used with every subject *except* third-person singular ("I bark", "they bark", but not "the dog bark"), so it carries vagr = n3sg. The sentence rule then demands

$$\texttt{<NP vagr>} = \texttt{<VP vagr>},$$

which for this input is the incompatible unification

$$\mathsf{3sg} \sqcup \mathsf{n3sg} = \top.$$

The other readings fare no better: the bare form fails the sentence rule's finiteness requirement, and the noun reading leaves a Det–N–N sequence that no rule licenses. Therefore strict parsing fails. But if the subject–verb agreement constraint is relaxed, the finite verbal parse completes, carrying that one violation. The engine reports:

```
the dog bark
        ^^^^


Verdict:   ungrammatical

Violation: subject-verb agreement (number/person)
Rule:      S -> NP VP
Fix:       "the dog barks"  |  "the dog barked"  |  "the dogs bark"
```

Each fix is a repair candidate the engine substituted and re-parsed; only candidates that parse cleanly are reported. The past-tense suggestion is there because English past-tense verbs do not show this agreement distinction at all, so `the dog barked` is as valid a one-word repair as `the dog barks`.

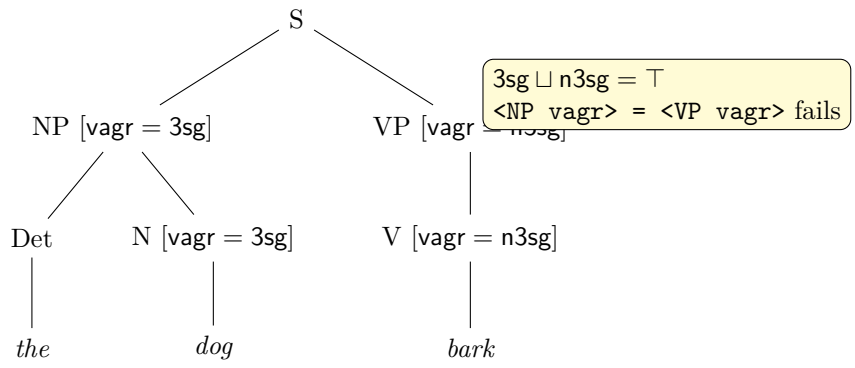

Figure 3: The relaxed parse tree for `the dog bark`. Feature values percolate up from the words: the NP is third-person singular because its head noun is; the VP is non-third-singular because the finite form `bark` is. The sentence rule's agreement equation cannot unify the two values, and that failed equation—rule, constituents, feature path, and both values—is exactly the content of the report.

The explanation is not a separate hand-written special case. It is the failed unification, viewed from the user's side.

16 Auxiliaries, clitics, and head projection

English auxiliaries show why morphology and syntax cannot be cleanly separated. In `I don't bark`, the token `n't` is part of the auxiliary structure. In `he doesn't bark`, subject agreement is carried by `does`, not by the lexical verb. In `he'll bark`, the modal `'ll` does not require the following verb to agree with the subject, but it does require that verb to appear in its base form.

The grammar therefore gives auxiliaries their own projection, AuxP, rather than treating them as ordinary lexical verbs. This has two advantages. First, auxiliary-specific constraints can be stated locally. Second, the parser does not accidentally merge the feature environment of an auxiliary with the feature environment of its complement verb.

The relevant distinction is not only agreement but *verb form*. English auxiliaries select particular forms of their complements: modals and do-support select a bare verb, perfect `have` selects a past

participle, and progressive `be` selects a present participle. The engine represents this selection with a feature

$$\text{VFORM} \in \{\text{BARE}, \text{FIN}, \text{PASTP}, \text{PRESP}\}.$$

Here bare covers base forms such as `bark`; FIN covers finite present and past forms; PASTP covers past participles; and PRESP covers present participles such as `barking`.

The value FIN is intentionally coarse. English finite verbs still vary internally: present-tense verbs show agreement distinctions, and past-tense verbs usually do not. Those facts are handled by agreement features and by the lexical analyses emitted by morphology. For auxiliary selection, however, the main question is simply whether a verb can head a finite clause. Collapsing the finite forms into one value lets a main-clause rule require finiteness with a single equation:

```
<VP vform> = fin.
```

English also contains substantial syncretism: one surface form may realize multiple grammatical forms. The engine handles this by ambiguity rather than by special cases. For example, `bark` may be analyzed both as a bare form and as a finite present form; `barked` may be analyzed both as a finite past form and as a past participle. The parser keeps all analyses whose features are compatible with the surrounding syntax and discards the rest.

Auxiliary selection is then just another feature equation:

```
AuxP -> Aux VP
    <VP vform> = <Aux req>   ! "wrong verb form after the auxiliary" fix: agree-verb
```

The auxiliary carries the verb form it requires in its `req` feature. Modals and do-support set `req = bare`; perfect `have` sets `req = pastp`; progressive `be` sets `req = presp`. The rule does not need separate procedural checks for each auxiliary. It only unifies the complement's `vform` with the auxiliary's requirement.

This rejects errors such as `I'll barked`. The clitic `'ll` is a modal auxiliary, so it requires a bare complement. The form `barked` can be finite past or past participle, but it is not bare, so the equation

```
<VP vform> = <Aux req>
```

fails. Because the diagnostic is attached to that equation, the report can name the error as a wrong verb form after the auxiliary and suggest the compatible base form: `I'll bark`.

17 Nominal and adjectival structure

The early examples in the engine focus on subject–verb agreement because it is the smallest diagnostic that demonstrates parsing plus unification. Expanding coverage quickly makes noun phrases more important. English noun phrases contain determiners, pronouns, adjectives, degree morphology, and stacked modifiers:

```
the big old dog,    the bigger dog,    he,    the dog.
```

The point of adding these constructions is not to introduce a new mechanism, but to test whether the same grammar architecture scales to more internal structure.

17.1 Adjectives and stacking

An attributive adjective modifies an NP-internal nominal projection without changing the head noun's agreement features. This means adjective stacking can be handled by recursive phrase structure while head features continue to percolate from the noun.

```
Nom -> Adj head:Nom
    <Nom agr> = <head agr>
```

17.2 Degree as a morphological feature

Comparative and superlative adjectives are still adjectives. The syntax does not need a separate category for `bigger` or `biggest`; morphology adds a degree feature:

$$\mathsf{degree} \in \{\mathsf{pos},\ \mathsf{comp},\ \mathsf{sup}\}.$$

Regular adjectives use a productive paradigm:

$$\texttt{big} \mapsto \texttt{bigger}, \texttt{biggest}, \qquad \texttt{happy} \mapsto \texttt{happier}, \texttt{happiest}.$$

Suppletive adjectives such as `good`, `better`, and `best` are lexical entries sharing a lemma rather than outputs of a spelling rule.

17.3 Pronouns, determiners, and possessives

Pronouns differ from ordinary nouns because they can project a complete noun phrase without a lexical noun. They also carry case directly:

$$\texttt{he} : \mathsf{Pron}[\mathsf{case} = \mathsf{nom}], \qquad \texttt{him} : \mathsf{Pron}[\mathsf{case} = \mathsf{acc}].$$

This lets the same VP-object rule reject `the cat chased I` and accept `the cat chased me`.

Possessive forms split into two syntactic types. A possessive pronoun is already an NP:

$$\texttt{mine},\ \texttt{yours},\ \texttt{hers}.$$

A possessive adjective, or possessive determiner, combines with a nominal head:

$$\texttt{my dog},\ \texttt{your cat},\ \texttt{his book}.$$

The surface similarity is misleading. `mine` can stand where an NP is required; `my` cannot. The grammar therefore treats possessive pronouns as pronouns and possessive adjectives as determiners. This is another case where the category assigned by the lexicon determines which phrase-structure rule the parser may use.

18 Trace mode and explainability

A useful engine needs to explain itself to two audiences: the user and the author. User-facing reports describe the violated grammatical rule. Trace mode exposes the internal computation that led there.

A trace records per-token morphology and the Earley chart operations: predictions, scans, completions, scan misses, and feature clashes. For an input such as `the dog bark`, a trace can show that `bark` was considered as both noun and verb, that the noun path died structurally, and that the verb path reached a subject–verb agreement clash.

This is the same principle at two scales. The user sees a grammatical explanation; the developer sees the parser state that produced it.

19 Performance and accidental complexity

The formal architecture of the engine is designed to keep ordinary analysis cheap. Morphology is finite-state, parsing is polynomial, and feature constraints are checked locally as rules are applied. In principle, then, a larger lexicon should not make every part of the system slower. Adding more words should increase the size of the morphology, but it should not make the parser repeatedly search the entire vocabulary.

The heavy benchmark was built to test exactly that assumption. It imported a 10k-word noun lexicon and measured four kinds of input: ordinary grammatical sentences, direct mal-rule matches, agreement errors diagnosed by constraint relaxation, and missing-determiner errors diagnosed by repair generation.

Input class	n	p50	p95	max
grammatical	42	4.64 ms	6.15 ms	9.92 ms
mal-rule	14	8.94 ms	11.78 ms	11.78 ms
relaxed S–V agreement	14	44.29 ms	49.27 ms	49.27 ms
relaxed missing determiner	14	75.07 ms	90.66 ms	90.66 ms

Table 4: Analyze-time before indexing on a 10k-word imported noun lexicon.

The result separated the ordinary path from the diagnostic path. Grammatical inputs and direct mal-rule matches stayed in the low-millisecond range while relaxed diagnostic recovery was much slower.

The cause was an accidental scaling cliff in lexical decoding. For each tag sequence, the morphology layer scanned every root in the lexicon to find stems compatible with the surface word. This made morphological analysis $O(R)$ per word, where R is the number of roots in the lexicon. On a small grammar, that cost was invisible. Inside diagnostic recovery, however, it was multiplied: each generated repair candidate triggered another analysis, and each analysis rescanned the full 10k-root lexicon.

The fix was to index roots by surface prefix, or equivalently to use the trie structure that already stores them. Candidate lookup then changes from a full scan over all roots to a walk over the possible prefixes of the input word.

The indexed walk preserves the old semantics. Overlapping stems such as `do` and `dog` are both found, each at its own prefix length. The prefix is built by concatenating emitted tape symbols rather than by indexing directly into the string, so multi-code-unit characters remain consistent with the representation used by the trie. The index is memoised on the parsed morphology object, which is immutable after grammar loading. The $O(R)$ construction cost is therefore paid once per grammar rather than once per token.

Indexing the roots closed the lexicon-dependent part of the cliff. Indexing the repair candidates then reduced the remaining recovery overhead. On the original heavy benchmark, overall p95 fell from 77 ms to 34 ms after root indexing, and then to 16.7 ms after fix indexing.

The grammar has since grown beyond subject–verb agreement to include adjectives, degree morphology, coordination, adverbs, pronouns, and the auxiliary system, and a third optimization round applied the same lesson to three more edges: the rewrite-cascade transducers were being re-inverted on every analyzed token (the inversion is now computed once per machine and cached), morphology was being run twice per sentence (once for the unknown-word check and once to build lexical items), and the feature-structure signatures used as chart-packing keys were being re-serialized on every chart operation (they are now memoised on the immutable structures). The benchmark on the current fragment, with the same 10k-word imported lexicon:

Input class	p50	p95
grammatical	0.3 ms	0.5 ms
grammatical (adjective)	0.2 ms	0.8 ms
grammatical (coordination)	0.3 ms	0.8 ms
mal-rule	0.5 ms	2.3 ms
relaxed S–V agreement	1.2 ms	2.5 ms
relaxed missing determiner	1.2 ms	1.9 ms
relaxed coordination	2.2 ms	4.2 ms

Table 5: Analyze-time on the current grammar with a 10k-word imported lexicon, after the third optimization round. Two things hold: the lexicon-size cliff stays closed, and the diagnostic path is now within a small multiple of the grammatical path. The worst observed call across the whole workload is 4.2 ms; the ≈30–40 ms first-call spikes seen in earlier rounds no longer appear after the subsequent optimization pass (cached transducer inversion, capped repair search, memoised chart keys).

The broader lesson is that accidental complexity often enters through the edges of an otherwise sound architecture, and it re-enters there too. The formal design made morphology regular, parsing polynomial, and diagnostics local, yet each measurement round found another routine quietly repeating work the model says is constant: a full-lexicon scan, then a per-token transducer inversion, then a duplicated morphology pass. Precomputing or caching the value keyed by the operation actually being performed removed each one without changing the linguistic model.

20 A second fragment: Swedish

The first real test of language neutrality is not whether the English grammar can grow. It is whether a second language can be added without adding a second engine. A grammar engine is language-neutral only if the reusable machinery remains fixed while language-specific facts are supplied as data: lexical entries, morphology, feature declarations, phrase-structure rules, tokenizer policy, and diagnostics.

This distinction is important. In multilingual NLP, Ploeger et al. argue that claims about generalization across languages require principled language sampling: it is not enough to evaluate on a convenient list of languages, because the goal is to cover typological variation rather than merely

increase the number of language names in an experiment [8]. Their point is methodological, but it applies here as well. A second grammar fragment is not proof that the engine is typologically general. It is a controlled stress test: does the architecture force English assumptions into the core, or can those assumptions be moved into grammar data?

Swedish is useful for this first test because it is close enough to English that the same broad pipeline applies, but different enough to exercise other parts of the system. It is still a Germanic language, so it is not a typologically broad sample in the sense of Ploeger et al. [8]. That limitation is useful to state explicitly. The Swedish fragment is not meant to prove universal language coverage. It is meant to show that the engine can host a second grammar without changing the parser, unifier, morphology compiler, or diagnostic machinery.

English makes subject–verb agreement central. Swedish does not: present-tense verbs do not agree with the subject in person or number. If subject–verb agreement were hard-coded into the parser, Swedish would have to disable an English assumption. Instead, Swedish simply does not write that equation. The absence of a constraint is ordinary grammar data, not an exception in the engine. In this sense, the parser does not know that English has subject–verb agreement or that Swedish lacks it. It only knows how to apply whatever feature equations the grammar supplies.

The nominal system pushes in the other direction. Swedish nouns distinguish common and neuter gender, and attributive adjectives agree with the noun they modify. Definiteness is also partly suffixal:

`hund` ‘dog’ ↦ `hunden` ‘the dog’, `hundar` ‘dogs’ ↦ `hundarna` ‘the dogs’.

In definite noun phrases, Swedish can also show double definiteness:

`den stora hunden` ‘the big dog’

This means the Swedish fragment tests morphology, feature percolation, and determiner–adjective–noun agreement in a place where English is comparatively poor. The same feature-structure machinery that checks English subject–verb agreement can check Swedish gender, number, and definiteness agreement; only the features and equations differ.

The important point is architectural. Swedish does not require a Swedish parser. It requires Swedish lexical entries, Swedish morphology, Swedish feature constraints, and a Swedish start/tokenizer declaration. The same Earley chart, unification engine, morphology compiler, and diagnostic machinery remain in place. The engine is therefore language-neutral at the level of machinery, while remaining language-specific at the level of grammar data.

This is also the right way to interpret future expansion. Adding more languages should not mean adding arbitrary examples until the project looks multilingual. A stronger test would choose languages because they stress different architectural assumptions. A language with rich case marking would test case features and freer word order. An agglutinative language would test long morphological chains. A language without whitespace word boundaries would test tokenization and segmentation. A non-concatenative morphology would test whether the morphology compiler is too biased toward suffixing and prefixing. This follows the same lesson as typologically diverse evaluation in multilingual NLP: breadth should be measured by structural variation, not only by the number of languages supported [8].

Swedish is therefore a first smoke test, not a final claim. It shows that one English-specific pressure point—subject–verb agreement—is not built into the engine, and that another language can introduce different agreement and morphological facts without changing the core. The next fragments

should be chosen not merely because they are interesting, but because each one asks a new question of the architecture.

21 The grammar file format

The previous sections describe the engine abstractly: morphology is finite-state, syntax is parsed over a context-free backbone, and grammatical constraints are checked by unifying feature structures. The `.gram` file is where those ideas become executable data. It is the engine's source language.

This is the compiler analogy at its most literal. The engine reads grammars from plain-text `.gram` files. The format is a small declarative language: lexicon entries define words, phrase rules define structure, feature equations define constraints, and directives configure morphology and modular loading. Its lexicon entries introduce symbols, its phrase-structure rules describe admissible local trees, its equations impose feature constraints, and its diagnostic annotations say which failures are meaningful enough to report. Loading a grammar therefore resembles compiling a source file: includes and imports are resolved, declarations are checked, morphology is compiled, and ill-typed feature values are rejected before analysis begins.

The format is PATR-II [11] in spirit, following the unification-grammar tradition, but extended with the pieces this engine needs for grammatical diagnosis: feature declarations, morphology directives, diagnostic annotations, and mal-rules for known wrong patterns.

There are four main families of statements:

1. lexicon entries, which attach categories and features to surface forms;
2. phrase-structure rules, which build constituents and impose equations;
3. mal-rules, which recognize known ungrammatical patterns directly;
4. directives, which configure the grammar loader and morphology compiler.

Whitespace separates tokens but is otherwise insignificant. A `#` character begins a comment running to the end of the line, and blank lines are ignored. The grammar of the format is as follows, where `::=` defines a nonterminal, `|` marks alternatives, `{}` means zero or more repetitions, `[]` marks an optional part, and `EOL` is the end of a line.

```
grammar     ::= { lexentry | rule | malrule | directive }

lexentry    ::= word ":" cat { featpath "=" value } EOL

rule        ::= production { equation }
production  ::= cnst "->" cnst { cnst } EOL
cnst        ::= [ alias ":" ] sym
                # a constituent: a category, optionally named by an alias

equation    ::= rulepath "=" ( rulepath | value ) [ diag ] EOL
diag        ::= "!" string [ "fix:" strategy { "|" strategy } ]

malrule     ::= "%mal" production "*ERR" string
```

```
directive   ::= feature | class | replrule | include | import
            | start | tokenizer | clitic

start       ::= "%start" sym EOL
tokenizer   ::= "%tokenizer" name EOL
clitic      ::= "%clitic" token { token } EOL

feature     ::= "%feature" feature ":" ( value { "|" value } | "*" ) EOL
class       ::= "%class" name ":" sym { "|" sym } EOL
replrule    ::= "%rule" pat ":" pat [ "=>" { ctxsym } "_" { ctxsym } ] EOL
include     ::= "%include" path EOL
import      ::= "%import"  path EOL

ctxsym      ::= sym | name

featpath    ::= "<" feature { feature } ">"
rulepath    ::= "<" name { feature } ">"

word        ::= token
cat, sym    ::= identifier
alias, name ::= identifier
feature     ::= identifier
value       ::= identifier
strategy    ::= identifier
string      ::= '"' { char } '"'
```

The syntax is intentionally small, but several semantic conventions are doing important work.

First, `=` is unification, not assignment. An equation such as `<NP agr> = <VP agr>` does not copy a value from one side to the other. It declares that the two feature structures must be compatible. When a rule is applied, all of its equations are unified together. If every merge succeeds, the rule builds a constituent with the resulting information. If any merge conflicts, the rule application fails and that parse branch dies. This is the same operation described in Section 6, now exposed as grammar syntax.

Second, paths name pieces of feature structure. In a lexicon entry, the constituent is implicit: the path refers to the entry's own feature structure.

```
dog : N <num>=sg <pers>=3
```

This says that `dog` is a noun whose number is singular and whose person is third. In a phrase-structure rule, the first name inside the angle brackets selects one of the constituents in the production, and the remaining names walk into that constituent's feature structure.

```
S -> NP VP
  <NP agr> = <VP agr> ! "subject-verb agreement" fix: agree-verb
```

Here the rule builds a sentence from a noun phrase and a verb phrase, but only when their agreement features unify. The diagnostic annotation says that if relaxing exactly this equation rescues the parse, the failure should be reported as a subject-verb agreement error.

Third, omitted information is not false information. A missing feature is unconstrained and unifies

with anything. This matters because many grammatical facts are best expressed by silence. For example, if a past-tense English verb does not participate in present-tense agreement, the grammar does not need to write a special value meaning "irrelevant"; it can simply omit the agreement feature.

Fourth, a `%mal` rule recognizes an ungrammatical pattern directly. A diagnostic annotation on an ordinary rule says, "this constraint may explain a failure." A mal-rule says, "this wrong structure is known and should always be reported when matched." The two mechanisms complement each other: relaxation can discover local constraint failures, while mal-rules cover recurrent errors that are easier to recognize as patterns.

Directive lines are discharged before ordinary parsing begins. The loader expands `%include` and `%import`, checks `%feature` declarations, records `%class` symbol sets for morphology rules, and adds `%rule` declarations to the morphophonemic rewrite cascade. This front-end pass is why many grammar mistakes are caught at load time rather than turning into silent parse failures later.

The start symbol is also grammar data. Earlier versions of the engine assumed that every grammar began at `S`. That was harmless while there was only one English fragment, but it made the parser less language-neutral than the rest of the pipeline. The directive

```
%start S
```

declares the root category explicitly. If it is omitted, the loader defaults to the left-hand side of the grammar's first rule, so a grammar that never mentions `S` still gets a usable entry point. If it is present, the symbol must name a real nonterminal. This turns the parser's entry point from an engine constant into a checked part of the grammar.

Tokenization is configured in the same spirit. The directive

```
%tokenizer whitespace
%clitic n't 're 've 'll 'm 'd 's
```

selects the whitespace tokenizer and gives it the clitic inventory for English. The tokenizer is therefore still simple, but it is no longer hardcoded. The Swedish or Russian grammar can use the same whitespace tokenizer without inheriting English clitic splitting, and a future grammar can fail loudly if it names a tokenizer that the engine does not provide.

21.1 Multi-file grammars: `%include` and `%import`

Two directives manage multi-file grammars.

`%include path` Textually includes another `.gram` file at the point of the directive, as if its contents appeared inline. Rules, lexicon entries, and feature declarations in the included file are resolved in the including grammar's namespace.

`%import path` Loads an external lexicon file—typically a tab-separated table of stems and paradigm names—as additional morphological data. Unlike `%include`, an imported file is not a `.gram` source: it does not contain rules or directives, only stem entries that are appended to the morphology's root table.

A concrete example: the English grammar includes shared closed-class entries from a common `closed-class.gram` file and imports a large open-class noun lexicon from `nouns.tsv`:

```
%include shared/closed-class.gram
```

```
%import  data/nouns.tsv
```

`%include` operates at the source level: the loader splices the referenced file's lines into the stream at the point of the directive, exactly as if those lines appeared inline. Cyclic includes are detected via a recursion stack: the same file may be included from two unrelated places, but a chain that would re-enter a file already on the stack is rejected as a load error.

`%import` does not splice `.gram` source. Instead the loader reads the referenced file as a tab-separated table and synthesizes a `%lex Root` block from its rows. The TSV format is:

```
stem TAB paradigm [TAB <feat>=val ...]
```

Column 1 is the surface stem, column 2 is the continuation lexicon (paradigm) name, and the remaining columns supply additional feature declarations using the same `<path>=value` syntax as the grammar. Comment lines beginning with `#` and blank lines are ignored. No header row is required.

21.2 Load-time checking

Several classes of grammar mistake are caught during loading rather than during analysis. Representative diagnostics include:

Undeclared feature value A lexicon entry or equation uses a value not listed in the corresponding `%feature` declaration. This prevents silent mismatch: a typo in a feature value is caught before it can silently fail to unify with anything.

Undefined nonterminal A phrase-structure rule references a category that never appears on any rule's left-hand side and has no lexicon entries. This catches rules that reference an absent category.

Ambiguous constituent name A rule contains two daughters of the same category without aliases, and a feature equation references that category by name alone. This is the constituent-addressing error described in Section 21.3; it is a load-time error rather than a silent runtime conflict.

Missing start symbol The symbol named by `%start` does not appear on any rule's left-hand side.

Cyclic include A chain of `%include` directives forms a cycle.

Every error carries an origin label of the form `file:line`, including across `%include` boundaries, so load errors point at the originating file and line even in a multi-file grammar. The three error types thrown are `grammar_error` (malformed grammar syntax), `load_error` (file-directive failure), and `feature_error` (feature validation); all extend `Error` and carry the origin in their message. Representative messages:

`unknown feature 'nummm' ... in lexicon entry 'dog'` A feature name not declared with `%feature`.

`feature 'num' has undeclared value 'singular' (allowed: sg | pl) in ...` A value outside the closed value domain.

`ambiguous constituent 'NP' (matches 2 daughters); give it an alias` A bare category name used in an equation when two daughters share that category.

`unknown constituent 'Obj'` An equation references a name that is neither an alias nor a category in the production.

`circular %include: morph.gram` A `%include` chain would re-enter a file already being expanded.

`%start S is not a nonterminal` The symbol named by `%start` has no rule defining it.

`TSV row needs at least <stem> TAB <paradigm>: ...` A malformed row in a file loaded by `%import`.

21.3 Constituent addressing

A subtle issue appears once a production contains two constituents of the same category. In the early grammar format, a path such as `<NP num>` named a constituent by its category symbol. That works for a rule like

```
S -> NP VP
```

because there is only one noun phrase. It fails for rules such as coordination or ditransitives:

```
NP -> NP Conj NP
VP -> V NP NP
```

In those productions, the category name `NP` is no longer enough to identify a unique constituent. A symbol-keyed environment silently conflates the two daughters, so one noun phrase can overwrite the other.

The current format solves this by giving every constituent occurrence a distinct identity. A production may also assign aliases to constituents, and equations may refer to those aliases:

```
NP -> left:NP Conj right:NP
  <left num> = <right num>

VP -> give:V obj1:NP obj2:NP
  <obj1 case> = acc
  <obj2 case> = acc
```

Name resolution happens at load time. A name in a rule path is resolved in this order:

1. a declared alias, if one exists;
2. the left-hand-side category, which always names the mother;
3. a daughter category, but only if that category occurs exactly once.

If a category occurs more than once among the daughters, it must be aliased. Using the bare category name is a load-time error. This makes ambiguous grammar rules fail early, before they can produce incorrect parse environments.

This addressing scheme is separate from head-feature inheritance. When the left-hand-side category occurs exactly once among the daughters, the mother may default to that daughter's structure. Thus a rule such as

```
NP -> NP PP
```

can still percolate the head noun phrase without writing explicit equations. That inheritance convention is not an addressing rule; it is a linguistic default. This distinction is why auxiliary projections remain useful. A modal auxiliary should not inherit the agreement requirements of its complement verb, so the grammar gives it a distinct projection such as `AuxP`.

22 Scope, limitations, and readiness for expansion

The current system contains small English and Swedish fragments with plans to expand to Russian and Japanese, not wide-coverage grammars. It is intentionally conservative: it reports only errors it can localize through declared constraints, relaxed parses, or mal-rules. This gives up coverage in exchange for trustworthiness. A missing parse is not treated as proof of a mistake unless the engine can point to the rule that failed.

The last architecture-level blocker before grammar growth—addressing constituents in productions that repeat a category—is resolved at the format level (Section 21.3): repeated categories require aliases, and ambiguous bare names fail at load time, so the grammar can grow without later rewriting rules to distinguish same-category daughters.

The first expansions beyond subject–verb agreement were chosen to stress different parts of the pipeline. Pronouns test case features and noun-phrase projection. Adjectives test recursive modifier structure and head-feature percolation. Comparative and superlative forms test productive morphology, spelling rules, and suppletion. Adverbs test modifier attachment outside the noun phrase. Coordination tests repeated-category addressing and agreement across parallel structures.

Together, these additions check that the engine is not a collection of isolated grammar checks. Each new construction enters through the same pipeline: morphology proposes analyses, the parser builds structure, feature equations enforce constraints, and diagnostics are reported only when a named failure can be localized.

The next expansion risk is therefore no longer the basic representation, but ambiguity management. As the grammar gains more modifiers, attachment sites, and lexical categories, the parser will naturally produce more competing analyses. The engine must continue to preserve useful ambiguity while suppressing spurious diagnostics. That is the main correctness pressure on future lexicon and rule growth.

23 Evaluation

The preceding sections make structural claims: the engine reports only errors it can localize through a named constraint, a matched mal-rule, or a successful constraint relaxation. Verifying those claims empirically requires labeled data and a breakdown of the engine's behavior into three outcomes: *accept* (grammatical), *flag* (ungrammatical, with a named constraint), and *abstain* (no analysis or unknown word).

The relevant metrics for a diagnostic engine differ from standard GEC metrics. In addition to detecting errors, the engine must correctly *abstain* on inputs outside its declared coverage and must correctly name the violated constraint rather than merely flagging the sentence as wrong. Two instruments measure this: an engineered regression corpus that pins down what the grammar claims to cover, and a held-out probe set that measures how the engine behaves on input it was never tuned to.

Regression corpus. The engine ships with a labeled regression corpus (`test/corpus.txt`) covering the English fragment. Each case carries a verdict label and, for ungrammatical cases, a detail string that the reported violation's message or rule must match as a substring. The corpus is the engine's correctness net: all cases pass under the current grammar.

Category	Detail	Cases
Grammatical	—	74
Ungrammatical (total: 30)		
Subject–verb agreement	agreement	9
Article agreement (a/an)	article	8
Case errors	nominative/accusative	6
Wrong verb form after aux	verb form	4
Number agreement	number	1
Word-order mal-rule	word order	1
Morphology mal-rule	irregular plural	1
No analysis (out of coverage)	—	2
Unknown word	—	3
Total		109

Table 6: English regression corpus (`test/corpus.txt`). All 109 cases pass under the current grammar. For each ungrammatical case, the detail column names a substring that the engine's reported violation message or rule must contain, verifying not only the verdict but the explanation. A Swedish corpus (`test/corpus.sv.txt`, 77 cases) follows the same format and exercises the Swedish fragment's gender, number, and definiteness constraints.

This corpus is an *engineered* regression suite. Every ungrammatical case was written to exercise a specific declared constraint, so recall is 100% by construction within the declared scope, and the suite says nothing about behavior on input the grammar was not built around. That question needs a second instrument.

23.1 A held-out probe set

The probe set (`test/eval.en.txt`, run by `scripts/eval.ts`) contains 206 sentences written against the vocabulary list alone—without consulting the rule inventory—and labeled for gold grammaticality before being run. Its composition targets the four situations a deployed diagnostic engine meets:

- **good** (100): grammatical sentences across the fragment's constructions, including two deliberately hard probes (a nested auxiliary chain and a bare-noun idiom);
- **bad, in scope** (61): ungrammatical sentences whose error types fall within the declared constraints, each with the constraint it should name;
- **bad, out of scope** (30): genuinely ungrammatical sentences whose error types were never declared (argument structure, doubled determiners, fragments, stacked finite auxiliaries)—the correct behavior is abstention, since no rule names the error;
- **out of vocabulary** (15): sentences using words the lexicon lacks.

Gold label	n	accept	flag	abstain
good	100	98.0%	2.0%	0.0%
bad, in scope	61	0.0%	100.0%	0.0%
bad, out of scope	30	26.7%	6.7%	66.7%
out of vocabulary	15	0.0%	0.0%	100.0%

Table 7: Probe-set behavior by gold label. Headline metrics: precision of the `ungrammatical` verdict 96.9% (63 of 65 flags are real errors); recall on in-scope errors 100% with 100% diagnosis accuracy (every flagged in-scope error names the intended constraint); abstention on out-of-scope errors 66.7%; every out-of-vocabulary sentence is routed to `unknown-word`.

The deviations are more instructive than the aggregates.

Two false positives, both predicted coverage gaps. The 2% false-positive rate comes entirely from the two deliberately hard probes. `he will have walked` is flagged because the auxiliary system is not recursive: an auxiliary phrase is finite by construction, so a second auxiliary under `will` fails the verb-form constraint. `he walks to school` is flagged because the grammar does not model bare-singular idioms (`to school`, `at home`) and demands a determiner. Both are coverage gaps that surface as *confident misdiagnoses*—the most expensive failure mode for a trust-first design, and exactly what a held-out probe set exists to find. Both are now known, named, and reproducible.

Overgeneration is concentrated in argument structure. Eight of the thirty out-of-scope errors are wrongly accepted, and seven of the eight share one cause: the lexicon does not record transitivity, so a missing object (`the dog wants`) or a spurious one (`the dog barks the cat`) parses cleanly. The eighth (`he is barks`) sneaks through on a plural-noun reading of `barks`. Declaring subcategorization as an ordinary feature (`<val>=tr|intr`) and constraining it in the VP rules would close this family without new machinery; it is the clearest next increment the probe set identifies. The two out-of-scope errors that were flagged rather than abstained on (`he will would walk`, `I am is happy`) are stacked finite auxiliaries caught by the verb-form constraint: the verdict is right and the named constraint is adjacent to, though not identical with, the real error.

What this evaluation is and is not. The probe set is held out with respect to the *rule inventory*, not with respect to the author: the sentences were written by the grammar's author against the vocabulary list, and in-vocabulary by construction. It therefore measures the engine's behavior on untuned input shapes, not its coverage of naturally occurring error distributions. The genuine blind evaluation remains future work: annotated learner text sampled from a corpus such as FCE or Lang-8, with the abstention rate on out-of-grammar errors tracked separately from true negatives. The probe set's contribution is narrower but real: it quantifies the design's central promise—flags are precise (96.9%), abstention does the work coverage gaps would otherwise push onto false claims—and it converts two latent coverage gaps into named, reproducible findings.

24 Conclusion

gram.en treats natural-language grammar checking as a compiler-front-end problem. The formal language theory explains the computational boundaries: regular methods are ideal for morphology,

context-free parsing is the practical syntactic baseline, and unrestricted grammar is too powerful to decide. Feature structures and unification add the local constraints needed for agreement, case, and selection. Mal-rules and constraint relaxation turn failures into named, localized diagnostics while respecting the coverage trap.

The resulting engine is deliberately limited, but its limitations are part of the design. It does not try to model all of natural language. It asks how much useful grammatical diagnosis can be obtained from explicit, inspectable rules. Within that scope, the verdict and the explanation are not separate products. They are the same computation viewed from two sides.

That property is also what gives the engine a role alongside, rather than against, modern neural systems. A judgment that carries its own constraint, span, and verified repair is a structured object other systems can consume: a verifier for a model's proposed correction, a reproducible evaluation criterion, a generator of labeled error data. The held-out evaluation in Section 23.1 is a small demonstration of the same point turned inward—because the engine's claims are explicit, its precision and its abstentions can be measured, and its failures arrive as named, reproducible findings rather than as silent drift.

References

[1] Christopher Bryant, Mariano Felice, Øistein E. Andersen, and Ted Briscoe. The BEA-2019 shared task on grammatical error correction. In *Proceedings of the Fourteenth Workshop on Innovative Use of NLP for Building Educational Applications*, pages 52–75, Florence, Italy, 2019.

[2] Ann Copestake. *Implementing Typed Feature Structure Grammars*. CSLI Publications, Stanford, CA, 2002.

[3] Jay Earley. An efficient context-free parsing algorithm. *Communications of the ACM*, 13(2):94–102, 1970.

[4] Ronald M. Kaplan and Martin Kay. Regular models of phonological rule systems. *Computational Linguistics*, 20(3):331–378, 1994.

[5] Robert T. Kasper and William C. Rounds. A logical semantics for feature structures. In *Proceedings of the 24th Annual Meeting of the Association for Computational Linguistics*, pages 257–266, 1986.

[6] Marcin Miłkowski. Developing an open-source, rule-based proofreading tool. *Software: Practice and Experience*, 40(7):543–566, 2010.

[7] Hwee Tou Ng, Siew Mei Wu, Ted Briscoe, Christian Hadiwinoto, Raymond Hendy Susanto, and Christopher Bryant. The CoNLL-2014 shared task on grammatical error correction. In *Proceedings of the Eighteenth Conference on Computational Natural Language Learning: Shared Task*, pages 1–14, Baltimore, Maryland, 2014.

[8] Esther Ploeger, Wessel Poelman, Andreas Holck Høeg-Petersen, Anders Schlichtkrull, Miryam de Lhoneux, and Johannes Bjerva. A principled framework for evaluating on typologically diverse languages. *Computational Linguistics*, 2025.

[9] Carl Pollard and Ivan A. Sag. *Head-Driven Phrase Structure Grammar*. University of Chicago Press, Chicago, 1994.

[10] Aarne Ranta. Grammatical framework: A type-theoretical grammar formalism. *Journal of Functional Programming*, 14(2):145–189, 2004.

[11] Stuart M. Shieber. The design of a computer language for linguistic information. In *Proceedings of the Tenth International Conference on Computational Linguistics (COLING)*, pages 362–366, 1984.

[12] Stuart M. Shieber. Evidence against the context-freeness of natural language. *Linguistics and Philosophy*, 8(3):333–343, 1985.

A Pumping lemmas: statements and proofs

The main text uses the pumping lemmas informally, as facts about dependency shapes. This appendix gives the formal statements and proofs. The strategy is proof by contradiction throughout: assume the language belongs to the class; every long enough string in it must then be pumpable; choose a string whose dependency structure cannot survive pumping. If every legal decomposition breaks the language, the language could not have belonged to the class.

Lemma A.1 (Regular pumping)**.** *If L is regular, then there is a pumping length $p \geq 1$ such that every string $s \in L$ with $|s| \geq p$ can be written $s = xyz$, where $|xy| \leq p$, $|y| \geq 1$, and $xy^iz \in L$ for every $i \geq 0$.*

Proof sketch. Let p be the number of states in a deterministic finite automaton for L. During the first p input symbols of any sufficiently long accepting run, the automaton visits $p + 1$ states (counting the start state). By the pigeonhole principle, some state repeats. The substring read between the two visits forms a loop, and the automaton can traverse that loop any number of times while still reaching an accepting state. □

Proposition A.2. *The language $\{a^nb^n \mid n \geq 0\}$ is not regular.*

Proof. Assume it is regular and let p be its pumping length. Choose $s = a^pb^p$. By the pumping lemma, $s = xyz$ with $|xy| \leq p$, $|y| \geq 1$, and xy^iz in the language for every $i \geq 0$. But $|xy| \leq p$ forces y to contain only a's. Pumping it up gives xy^2z, which has more a's than b's and is therefore not in the language. This contradiction shows that $\{a^nb^n \mid n \geq 0\}$ is not regular. □

Lemma A.3 (Context-free pumping)**.** *If L is context-free, then there is a pumping length $p \geq 1$ such that every string $z \in L$ with $|z| \geq p$ can be written $z = uvwxy$, where $|vwx| \leq p$, $|vx| \geq 1$, and $uv^iwx^iy \in L$ for every $i \geq 0$.*

Proof sketch. In a sufficiently tall parse tree, some nonterminal must repeat along a path from the root to a leaf. The portion of the tree between the two occurrences can be duplicated or deleted. This pumps two substrings, v and x, together, since both substrings derive from the repeated nonterminal's position in the tree. □

Proposition A.4. *The language $\{a^nb^nc^n \mid n \geq 0\}$ is not context-free.*

Proof sketch. Assume the language is context-free and let p be its pumping length. Choose $z = a^pb^pc^p$. In any legal decomposition $z = uvwxy$, the substring vwx has length at most p, so it can span at most two of the three blocks. Pumping v and x therefore changes at most two of the three

counts, leaving the equality of all three counts broken. The pumped string is not in the language, which contradicts the context-free pumping lemma. □

Proof architecture: Shieber's non-context-freeness result

Context-free languages are closed under intersection with regular languages and under homomorphism. Shieber [12] isolates a Swiss German fragment D such that, for a suitable regular language R and homomorphism h,

$$h(D \cap R) = \{\, a^m b^n c^m d^n \mid m, n \geq 0 \,\}.$$

The shape $\{a^m b^n c^m d^n \mid m, n \geq 0\}$ is not simple nesting: the first block must match the third, while the second block must match the fourth. The dependencies cross rather than close in last-in-first-out order. Since the resulting language is not context-free, and context-free languages are closed under the operations used to obtain it, D cannot have been context-free.

B Grammar syntax cheat-sheet

A compact reference for `.gram` file syntax.

Syntax	Meaning
Lexicon entries	
`word : Cat`	Assign category `Cat` to `word`.
`word : Cat <f>=v`	As above, with feature `f` set to value `v`.
Phrase-structure rules	
`A -> B C`	`A` rewrites as `B` followed by `C`.
`A -> lft:B C`	Same, with daughter `B` aliased as `lft`.
`<A f> = <B f>`	Equation: the `f` features of `A` and `B` must unify.
`<A f> = v`	Equation: `A`'s feature `f` must equal value `v`.
`... !"msg" fix: s`	Diagnostic: if relaxing this equation rescues a parse, report `msg` and apply strategy `s`.
Mal-rules	
`%mal A -> B C`	Recognize this ungrammatical pattern directly.
`*ERR "msg"`	Error message to report (fixes are derived: matched-constituent reorderings that parse cleanly).
Directives	
`%start S`	Declare `S` as the root category.
`%feature f : v1 | v2`	Declare feature `f` with closed value domain.
`%feature f : *`	Declare feature `f` with open value domain.
`%class Name : a | b`	Declare a character class for morphology rules.
`%rule A:B => L _ R`	Morphophonemic rewrite rule with context.
`%lex Name : Cat`	Declare a continuation lexicon (paradigm).
`%tokenizer name`	Select a tokenizer by name.
`%clitic t1 t2`	Declare clitic strings for the tokenizer.
`%include path`	Textually include another `.gram` file.
`%import path`	Import an external stem table.
General	
`#`	Comment to end of line.
`=`	Unification (not assignment).
`<A f g>`	Feature path: walk into `A`'s structure via `f`, then `g`.

Table 8: Quick reference for `.gram` file syntax.

C Glossary

Terminal A symbol that may appear in the final generated string: in practice a word, token, or punctuation mark. Derivation stops once only terminals remain.

Nonterminal An abstract syntactic category such as S, NP, or V. Nonterminals do not appear in final strings; they name intermediate structure that must be expanded by grammar rules.

Derivation A sequence of rule applications that rewrites the start symbol into a terminal string. A derivation is a proof that the string belongs to the language.

Finite-state transducer (FST) A finite automaton extended with an output tape. Each arc reads a symbol from the input and writes a symbol to the output. The engine uses FSTs for morphological analysis: an FST maps a surface word form to its possible lemma-plus-features

analyses.

Feature structure A record of grammatical properties carried by a word or phrase. At its simplest, it is a set of (feature, value) pairs such as [NUM : SG, PERS : 3]. The engine stores feature structures as partial maps from feature names to atomic values.

Unification The operation that merges two compatible feature structures into one. It succeeds when the two structures agree on every feature they both specify; it fails (returning $\top$) when they assign conflicting values to the same feature.

Context-free grammar (CFG) A grammar in which every rewrite rule has a single nonterminal on its left-hand side. CFGs can describe nested structure such as noun phrases inside verb phrases, and are recognized in $O(n^3)$ time. The engine's phrase-structure backbone is a CFG.

Earley parsing An algorithm that recognizes any context-free grammar in $O(n^3)$ time (and $O(n^2)$ for unambiguous grammars) by maintaining a chart of partial parses. The chart is well-suited to ambiguity and to attaching diagnostic information at the point of failure.

Mal-rule A grammar rule written to match an ungrammatical pattern directly, with an attached error message and repair suggestion. Mal-rules fire when the engine recognizes a known wrong structure.

Constraint relaxation A diagnostic strategy in which the engine reparses with every diagnostic-tagged equation allowed to record a violation instead of failing the parse. The full parse with the fewest recorded violations wins, and each of its violations is reported as a named, localized constraint failure.